Travaux pratiques – Serveurs Linux

Objectifs

Au cours de ces travaux pratiques, vous utiliserez la ligne de commande Linux pour identifier les serveurs exécutés sur un ordinateur donné.

Partie 1 : Serveurs

Partie 2: Utilisation de Telnet pour tester les services TCP

Matériel conseillé

- Poste de travail CyberOps VM

Instructions

Partie 1 : Serveurs

Les serveurs sont principalement des programmes écrits pour fournir des informations spécifiques sur demande. Les clients, qui sont aussi des programmes, communiquent avec le serveur, lui envoient la requête et attendent sa réponse. Nombreuses sont les technologies de communication client/serveur qui peuvent être utilisées, mais les réseaux IP sont les plus courants. Ces travaux pratiques se concentrent sur les clients et les serveurs basés sur des réseaux IP.

Étape 1: Accédez à la ligne de commande.

a. Connectez-vous au poste de travail virtuel CyberOps avec l'identifiant **analyst** et le mot de passe **cyberops**. Le compte **analyst** servira d'exemple de compte utilisateur pendant l'intégralité des travaux pratiques.

b. Pour accéder à la ligne de commande, cliquez sur l'icône **terminal** située sur le dock, en bas de l'écran de la machine virtuelle. L'émulateur de terminal s'ouvre.



Étape 2: Affichez les services en cours d'exécution.

Il est possible d'exécuter un grand nombre de programmes différents sur un ordinateur donné, en particulier un ordinateur exécutant un système d'exploitation Linux. De nombreux programmes s'exécutent en arrière-plan, il est donc possible que les utilisateurs ne détectent pas immédiatement ceux qui s'exécutent sur leur ordinateur. Sous Linux, les programmes en cours d'exécution sont aussi appelés *processus*.

Remarque : la sortie de votre commande **ps** sera différente, car elle dépendra de l'état de votre poste de travail virtuel CyberOps.

a. Utilisez la commande **ps** pour afficher tous les programmes exécutés en arrière-plan :

```
[analyst@secOps ~]$ sudo ps –elf
[sudo] password for analyst:
F S UID PID PPID C PRI NI ADDR SZ WCHAN STIME TTY TIME CMD
4 S root 1 0 0 80 0 - 2250 SyS_ep Feb27 ?        00:00:00 /sbin/init
1 S root 2 0 0 80 0 - 0 kthrea Feb27 ?        00:00:00 [kthreadd]
1 S root 3 2 0 80 0 - 0 smpboo Feb27 ?        00:00:00 [ksoftirqd/0]
1 S root 5 2 0 60 -20 - 0 worker Feb27 ?        00:00:00 [kworker/0:0H]
1 S root 7 2 0 80 0 - 0 rcu_gp Feb27 ?        00:00:00 [rcu_preempt]
1 S root 8 2 0 80 0 - 0 rcu_gp Feb27 ?        00:00:00 [rcu_sched]
1 S root 9 2 0 80 0 - 0 rcu_gp Feb27 ?        00:00:00 [rcu_bh]
1 S root 10 2 0 -40 - - 0 smpboo Feb27 ?        00:00:00 [migration/0]
1 S root 11 2 0 60 -20 - 0 rescue Feb27 ?        00:00:00 [lru-add-drain]
5 S root 12 2 0 -40 - - 0 smpboo Feb27 ?        00:00:00 [watchdog/0]
1 S root 13 2 0 80 0 - 0 smpboo Feb27 ?        00:00:00 [cpuhp/0]
5 S root 14 2 0 80 0 - 0 devtmp Feb27 ?        00:00:00 [kdevtmpfs]
1 S root 15 2 0 60 -20 - 0 rescue Feb27 ?        00:00:00 [netns]
1 S root 16 2 0 80 0 - 0 watchd Feb27 ?        00:00:00 [khungtaskd]
1 S root 17 2 0 80 0 - 0 oom_re Feb27 ?        00:00:00 [oom_reaper]
<some output omitted>
```

Pourquoi fallait-il lancer **ps** comme root (en ajoutant **sudo** au début de la commande)?

b. Sous Linux, les programmes peuvent également appeler d'autres programmes. La commande **ps** permet également d'afficher la hiérarchie de ces processus. Utilisez les options **—EJH** pour afficher l'arborescence des processus en cours d'exécution après le démarrage du serveur Web nginx avec des privilèges élevés.

Remarque : les informations de processus du service nginx sont mises en surbrillance. Vos valeurs PID seront distinctes.

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

```
[analyst@secOps ~]$ sudo /usr/sbin/nginx
[analyst@secOps ~]$ sudo ps –ejH
[sudo] password for analyst:
 PID PGID SID TTY TIME CMD
    1 1 1 ?        00:00:00 systemd
  167 167 167 ?        00:00:01 systemd-journal
  193 193 193 ?        00:00:00 systemd-udevd
  209 209 209 ?        00:00:00 rsyslogd
  210 210 210 ?        00:01:41 java
  212 212 212 ?        00:00:01 ovsdb-server
  213 213 213 ?        00:00:00 start_pox.sh
  224 213 213 ?        00:01:18 python2.7
  214 214 214 ?        00:00:00 systemd-logind
  216 216 216 ?        00:00:01 dbus-daemon
  221 221 221 ?        00:00:05 filebeat
  239 239 239 ?        00:00:05 VBoxService
  287 287 287 ?        00:00:00 ovs-vswitchd
  382 382 382 ?        00:00:00 dhcpcd
  387 387 387 ?        00:00:00 lightdm
  410 410 410 tty7 00:00:10 Xorg
  460 387 387 ?        00:00:00 lightdm
  492 492 492 ?        00:00:00 sh
  503 492 492 ?        00:00:00 xfce4-session
  513 492 492 ?        00:00:00 xfwm4
  517 492 492 ?        00:00:00 Thunar
 1592 492 492 ?        00:00:00 thunar-volman
  519 492 492 ?        00:00:00 xfce4-panel
  554 492 492 ?        00:00:00 panel-6-systray
  559 492 492 ?        00:00:00 panel-2-actions
  523 492 492 ?        00:00:01 xfdesktop
  530 492 492 ?        00:00:00 polkit-gnome-au
  395 395 395 ?        00:00:00 nginx
  396 395 395 ?        00:00:00 nginx
  408 384 384 ?        00:01:58 java
  414 414 414 ?        00:00:00 accounts-daemon
```

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

```
  418 418 418 ?        00:00:00 polkitd
<some output omitted>
```

Comment la hiérarchie des processus est-elle représentée par **ps**?

c. Comme mentionné précédemment, les serveurs sont principalement des programmes, souvent lancés par le système lui-même au moment du démarrage. La tâche exécutée par un serveur s'appelle un *service*. Dès lors, un serveur web fournit des services web.

La commande **netstat** est un excellent outil pour identifier les serveurs réseau exécutés sur un ordinateur. Tout l'intérêt de **netstat** repose sur sa capacité à afficher les connexions réseau.

Remarque : votre sortie peut être différente selon le nombre de connexions réseau ouvertes sur votre machine virtuelle.

Dans la fenêtre du terminal, saisissez **netstat**.

```
[analyst@secOps ~]$ netstat
Connexions Internet actives (sans serveurs)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 localhost.localdo:48746 localhost.local:wap-wsp ESTABLISHED
tcp 0 0 localhost.localdo:48748 localhost.local:wap-wsp ESTABLISHED
tcp6 0 0 localhost.local:wap-wsp localhost.localdo:48748 ESTABLISHED
tcp6 0 0 localhost.local:wap-wsp localhost.localdo:48746 ESTABLISHED
tcp6 0 0 localhost.local:wap-wsp localhost.localdo:48744 ESTABLISHED
tcp6 0 0 localhost.localdo:48744 localhost.local:wap-wsp ESTABLISHED
Processeurs de domaine UNIX actifs (sans serveurs)
Proto RefCnt Flags Type State I-Node Path
unix 3 [ ] DGRAM 8472 /run/systemd/notify
unix 2 [ ] DGRAM 8474 /run/systemd/cgroups-agent<some output omitted>
```

Comme illustré plus haut, **netstat** renvoie beaucoup d'informations lorsqu'il est utilisé sans options. Plusieurs options permettent de filtrer et de mettre en forme la sortie de **netstat**, afin de la rendre plus utile.

d. Utilisez **netstat** avec les options **–tunap** pour corriger la sortie de **netstat**. Notez que **netstat** permet de regrouper plusieurs options sous le même signe **-**.

Les informations relatives au serveur nginx sont mises en surbrillance.

```
[analyst@secOps ~]$ sudo netstat -tunap
[sudo] password for analyst:
Connexions Internet actives (serveurs et connexions établies)
```

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

```
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:6633 0.0.0.0:* LISTEN 257/python2.7
tcp 0 0 0.0.0.0:80 0.0.0.0:* LISTEN 395/nginx: master
tcp 0 0 0.0.0.0:21 0.0.0.0:* LISTEN 279/vsftpd
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 277/sshd: /usr/bin
tcp6 0 0 :::22 :::* LISTEN 277/sshd: /usr/bin
udp 0 0 192.168.1.15:68 0.0.0.0:* 237/systemd-network
```

Que signifient les options **–t**, **-u**, **–n**, **–a** et **–p** dans **netstat** ? (utilisez **man netstat** pour répondre)

L'ordre des options **netstat** est-il important ?

Les clients se connectent à un port et, en utilisant le protocole adapté, ils demandent des informations à un serveur. La sortie **netstat** ci-dessus affiche plusieurs services qui écoutent sur des ports spécifiques. Les colonnes intéressantes sont les suivantes :

- La première colonne indique que le protocole de couche 4 est en cours d'utilisation (UDP ou TCP, dans ce cas).
- La troisième colonne utilise le format **<ADDRESS:PORT>** pour afficher l'adresse IP locale et le port sur lequel un serveur spécifique est accessible. L'adresse IP 0.0.0.0 signifie que le serveur écoute au niveau de toutes les adresses IP configurées sur l'ordinateur.
- La quatrième colonne utilise le même format de processeur **<ADDRESS:PORT>** pour afficher l'adresse et le port de l'appareil à l'autre extrémité de la connexion. 0.0.0.0 :* signifie qu'aucun appareil distant n'utilise actuellement la connexion.
- La cinquième colonne affiche l'état de la connexion.
- La sixième colonne affiche l'ID du processus responsable de la connexion (PID). Il affiche également un nom court associé au processus.

En vous basant sur la sortie **netstat** indiquée au point (d), quel est l'état de la connexion du protocole de couche 4 et le PID du processus exécuté sur le port 80 ?

Bien que les numéros de port ne soient qu'une convention, selon vous, quel type de service s'exécute sur le port TCP 80 ?

e. Il est parfois utile de croiser les informations fournies par **netstat** avec **ps**. Après la sortie indiquée au point d, nous savons qu'un processus associé au **PID 395** est lié au port TCP 80. Le port 395 est utilisé dans cet exemple. Utilisez **ps** et **grep** pour répertorier toutes les lignes de la sortie **ps** qui contiennent **PID 395**: Remplacez 395 par le numéro PID pour votre instance en cours d'exécution particulière de nginx:

```
[analyst@secOps ~]$ sudo ps -elf | grep 395
```

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

```
[sudo] password for analyst:
1 S root 395 1 0 80 0 - 1829     19:33 ?        00:00:00 nginx: processus maître
/usr/bin/nginx
5 S http 396 395 0 80 0 - 1866   19:33 ?        00:00:00 nginx: worker process
0 S analyst 3789 1872 0 80 0 - 1190     19:53 pts/0 00:00:00 grep 395
```

Dans la sortie ci-dessus, la commande **ps** est transmise via la commande **grep** pour filtrer uniquement les lignes contenant le numéro 395. Il en résulte trois lignes dont le texte est renvoyé à la ligne.

La première ligne indique un processus qui appartient à l'utilisateur **root** (troisième colonne) et qui a été lancé par un autre processus avec le PID 1 (cinquième colonne), à 19:33 (douzième colonne)

La deuxième ligne indique un processus avec le PID 396, qui appartient à l'utilisateur **http** et qui a été lancé par le processus 395 à 19:33.

La troisième ligne illustre un processus avec le PID 3789 qui appartient à l'utilisateur **analyst** et qui a été lancé par un processus avec le PID 1872, comme la commande **grep 395**.

Le processus avec le PID 395 est **nginx**. Comment ce résultat a-t-il été obtenu à partir de la sortie ci-dessus ?

Qu'est-ce que **nginx** ? Quel est son rôle ? (Utilisez Google pour en savoir plus sur nginx)

La deuxième ligne indique que le processus 396 appartient à un utilisateur nommé http et qu'il est associé au numéro de processus 395 comme son processus parent. Qu'est-ce que cela signifie ? Ce comportement est-il habituel ?

Pourquoi la dernière ligne affiche-t-elle grep 395 ?

Partie 2 : Utilisation de Telnet pour tester les services TCP

Telnet est une application d'environnement à distance. Elle est considérée comme non sécurisée, car elle n'intègre pas de fonction de chiffrement. Les administrateurs qui choisissent d'utiliser Telnet pour gérer les serveurs et les périphériques réseau à distance exposent les identifiants de connexion sur ces serveurs, car les données de session sont transmises en texte clair. Bien que Telnet ne soit pas recommandé en tant qu'application d'environnement à distance, elle peut être très utile pour tester ou collecter des informations sur les services TCP rapidement.

Le protocole Telnet fonctionne sur le port 23 qui utilise TCP par défaut. Le client **telnet** permet toutefois de spécifier un autre port. En changeant de port et en se connectant à un serveur, le client **telnet** permet à un analyste de réseau de déterminer rapidement la nature d'un serveur spécifique en communiquant directement avec lui.

Remarque **: il est fortement recommandé d'utiliser** ssh **comme application d'environnement à distance au lieu de** telnet**.**

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

a. Dans la partie 1, **nginx** était en cours d'exécution et assigné au port TCP 80. Bien qu'une rapide recherche sur Google révèle que **nginx** est un serveur web léger, que peut faire un analyste pour s'en assurer? Que se passerait-il si un hacker remplaçait le nom d'un programme de malware par **nginx** pour faire croire aux utilisateurs qu'il s'agit du serveur web correspondant ? Utilisez **telnet** pour vous connecter à l'hôte local sur le port TCP 80 :

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Essai 127.0.0.1…
Connecté à 127.0.0.1.
Le caractère d'échappement est '^]'.
```

b. Appuyez sur plusieurs lettres du clavier. N'importe quelle touche fera l'affaire. Après avoir appuyé sur plusieurs touches, appuyez sur Entrée. Voici la sortie complète, y compris l'établissement de la connexion Telnet et les touches sélectionnées de manière aléatoire (fdsafsdaf, dans ce cas) :

```
fdsafsdaf
HTTP/1.1 400 Bad Request
Server: nginx/1.16.1
Date: Tue, 28 Apr 2020 20:09:37 GMT
Content-Type : text/html
Content-Length : 173
Connection : close

<html>
<head><title>400 Bad Request</title></head>
<body bgcolor="white">
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.16.1</center>
</body>
</html>
Connexion fermée par un hôte externe.
```

Grâce au protocole Telnet, une connexion TCP en texte clair a été établie par le client Telnet directement vers le serveur nginx, qui écoute 127.0.0.1 sur le port TCP 80. Cette connexion permet d'envoyer des données directement au serveur. Comme nginx est un serveur web, il ne comprend pas la séquence de lettres aléatoires qui lui a été envoyée et renvoie une erreur sous forme de page web.

Pourquoi l'erreur est-elle envoyée sous forme de page web ?

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco

Le serveur a signalé une erreur et a mis fin à la connexion, mais nous avons découvert des informations précieuses. Nous avons appris que :

1) Le processus **nginx** associé au PID 395 est en fait un serveur web.

2) The version of **nginx** is 1.16.1.

3) La pile réseau du poste de travail virtuel CyberOps est opérationnelle jusqu'à la couche 7.

Les services ne sont pas tous les mêmes. Certains sont conçus pour accepter les données non formatées et ne mettent pas fin au processus si des données incorrectes sont saisies sur le clavier. Voici un exemple :

c. Si vous analysez la sortie de **netstat** présentée précédemment, il est possible de voir qu'un processus est lié au port 22. Utilisez Telnet pour vous y connecter.

Le port TCP 22 est affecté au service SSH. Le service SSH permet à un administrateur de se connecter à un ordinateur distant en toute sécurité.

Voici la sortie :

```
[analyst@secOps ~]$ telnet 127.0.0.1 22
Essai 127.0.0.1…
Connecté à 127.0.0.1.
Le caractère d'échappement est '^]'.
SSH-2.0-OpenSSH_8.2
sdfjlskj
Invalid SSH identification string.
Connexion fermée par un hôte externe.
```

Utilisez Telnet pour vous connecter au port 68. Que se passe-t-il ? Expliquez votre réponse.

Questions de réflexion

1. Quels sont les avantages d'utiliser netstat ?
2. Quels sont les avantages d'utiliser Telnet ? Est-ce sûr ?

© 2018 - aa Cisco et/ou ses filiales. Tous droits réservés. Document public de Cisco